



Spring

Éléments courants

1

Spring

What Else



2

- ✓ Taches et Scheduling
- ✓ Méthodes Asynchrones
- ✓ FactoryBean
- ✓ Resource
- ✓ Evènements
- ✓ Internationalisation (multi langue)
- ✓ Caches / Redis
- ✓ Actuator / Prometheus

Tâches

- Vous pouvez utiliser le Spring comme ordonnanceur
- Il va fonctionner comme le CRON Unix ou \$U
- Spring utilise son propre framework mais il est aussi possible d'utiliser le framework Quartz
 - <http://www.quartz-scheduler.org/>

Tâches - **TaskExecutor**

- Pour lancer nos tâches nous devons choisir une implémentation d'un **TaskExecutor**
 - SyncTaskExecutor
 - SimpleAsyncTaskExecutor
 - ConcurrentTaskExecutor
 - ThreadPoolTaskExecutor
 - WorkManagerTaskExecutor
 - DefaultManagedTaskExecutor

Tâches - TaskExecutor

- **SyncTaskExecutor** : Cette implémentation exécute les appels de manière synchrone.
- Chaque invocation a lieu dans le thread appelant.
- Il est principalement utilisé dans les situations où le multi-threading n'est pas nécessaire, comme dans les cas de test simples

Tâches - TaskExecutor

- **SimpleAsyncTaskExecutor** : Cette implémentation ne réutilise aucun thread.
- Il démarre un nouveau thread pour chaque invocation.
- Cependant, il prend en charge une limite de simultanéité qui bloque toutes les invocations dépassant la limite jusqu'à ce qu'un emplacement soit libéré.
- C'est le premier niveau du pooling de thread (voir *ThreadPoolTaskExecutor*).

Tâches - TaskExecutor

- **ConcurrentTaskExecutor** : cette implémentation est un adaptateur pour une instance *java.util.concurrent.Executor*.
- Il existe une alternative (*ThreadPoolTaskExecutor*) qui expose les paramètres de configuration de l'*Executor* en tant que propriétés de bean.
- Il est rarement nécessaire d'utiliser directement *ConcurrentTaskExecutor*. Toutefois, si *ThreadPoolTaskExecutor* n'est pas suffisamment flexible pour répondre à vos besoins, *ConcurrentTaskExecutor* est une alternative.

Tâches - TaskExecutor

- **ThreadPoolTaskExecutor** : Il expose les propriétés du bean pour la configuration d'un *java.util.concurrent.ThreadPoolExecutor* et l'encapsule dans un TaskExecutor.
- Si vous devez vous adapter à un type différent de *java.util.concurrent.Executor*, nous vous recommandons d'utiliser plutôt un *ConcurrentTaskExecutor*.
- Implémentation qui sera utilisée par défaut par le Spring si aucune n'est explicitement déclarée.

Tâches - TaskExecutor

- **WorkManagerTaskExecutor** : Cette implémentation utilise CommonJ WorkManager comme fournisseur de service et constitue la classe idéale pour la configuration de l'intégration d'un pool de threads basé sur CommonJ sur WebLogic ou WebSphere dans un contexte d'application Spring.
- A réserver pour WebLogic ou WebSphere

Tâches - TaskExecutor

- **DefaultManagedTaskExecutor** : Cette implémentation utilise un *ManagedExecutorService* obtenu par JNDI dans un environnement d'exécution compatible JSR-236 (tel qu'un serveur d'applications Java EE 7+), remplaçant ainsi un CommonJ WorkManager.
- <https://www.jcp.org/en/jsr/detail?id=236>
 - Oracle GlassFish Server Open Source Edition 4.0
 - TmaxSoft TMAX JEUS 8
 - RedHat Wildfly 8.1.0
 - JBoss EAP 7.1
 - WebSphere 9
 - Weblogic 12.2.1

Tâches - TaskExecutor

➤ Déclaration d'un *TaskExecutor* en XML

```
<bean id="taskExecutor"
      class="org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor">
  <property name="corePoolSize" value="5"/>
  <property name="maxPoolSize" value="10"/>
  <property name="queueCapacity" value="25"/>
</bean>
```

➤ Déclaration d'un *TaskExecutor* en Code

```
@Bean
public TaskExecutor taskExecutor() {
    ThreadPoolTaskExecutor result = new ThreadPoolTaskExecutor();
    result.setCorePoolSize(5);
    result.setMaxPoolSize(10);
    result.setQueueCapacity(25);
    return result;
}
```

Tâches - TaskScheduler

- Pour organiser / ordonnancer nos tâches nous devons choisir une implémentation d'un **TaskScheduler**
 - **TimerManagerTaskScheduler** :
qui délègue à une instance CommonJ TimerManager, généralement configurée avec un annuaire JNDI.
 - **ThreadPoolTaskScheduler** : pour une utilisation plus standard, moins centralisée.
- **ThreadPoolTaskScheduler** sera l'implémentation qui sera utilisée par défaut par le Spring si aucune n'est explicitement déclarée.

Tâches - TaskScheduler

➤ Déclaration d'un **TaskScheduler** en XML

```
<bean id="threadPoolTaskScheduler"  
      class="org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler">  
  <property name="poolSize" value="5" />  
</bean>
```

➤ Déclaration d'un **TaskScheduler** en Code

```
@Bean  
public TaskScheduler taskScheduler() {  
  ThreadPoolTaskScheduler result = new ThreadPoolTaskScheduler();  
  result.setPoolSize(5);  
  return result;  
}
```

Tâches - Trigger

- En plus des deux éléments d'organisation précédents, il est possible de déclencher les tâches en fonction d'évènements
- Inspirée de JSR-236, L'idée de base du déclencheur est que les temps d'exécution peuvent être déterminés en fonction des résultats de l'exécution passée ou même de conditions arbitraires.
- L'objet **TriggerContext** nous permet de prendre en compte le résultat de l'exécution précédente

Tâches - Trigger

➤ L'interface **Trigger**

```
public interface Trigger {  
    Date nextExecutionTime(TriggerContext triggerContext);  
}
```

➤ L'interface **TriggerContext**

```
public interface TriggerContext {  
    Date lastScheduledExecutionTime();  
  
    Date lastActualExecutionTime();  
  
    Date lastCompletionTime();  
}
```

Tâches - Trigger

- Deux implémentations sont disponibles pour les Trigger :
 - **CronTrigger** : Il permet la planification de tâches en fonction d'expressions CRON.
 - **PeriodicTrigger** : qui accepte une période fixe, une valeur de délai initiale optionnelle et un booléen pour indiquer si la période doit être interprétée comme un taux fixe ou un délai fixe
- Ils seront généralement masqués par des annotations

Tâches - Scheduling

- Pour ordonnancer vos tâches, deux possibilités
 - Le faire à la main via les objets précédemment cités
 - Utiliser les annotations de méthodes :
 - ❑ **@Scheduled** : indique le paramétrage d'ordonnancement
 - ❑ **@Async** : indique que la méthode va se lancer de manière asynchrone
- **Attention** : comme toujours en Spring pour que les annotations fonctionnent il ne faudra pas oublier de les activer

Tâches - Scheduling

- **@Scheduled** : indique le paramétrage d'ordonnancement (porte les informations de la Trigger)
- Période et délai :
 - **Délai** : délai entre chaque exécution en ms. Du coup si la tâche prend du temps il est complexe de savoir quand sera enclenchée la suivante. Géré par le *Scheduler*.
 - **Période** : temps entre chaque lancement de la tâche indépendamment du temps que prend la tâche.

Tâches - Scheduling

➤ Exemple

```
@Scheduled(fixedDelay = 1000)
public void faireQuelqueChoseFixedDelay() {
    DireBonjour.LOG.info("Hello");
    // action qui dure 3mn
}
```

- Le prochain lancement de la tâche se fera
 - ➡ $3\text{mn} + 1\text{s} = 3\text{mn et } 1\text{s}$

Tâches - Scheduling

➤ Exemple

```
@Scheduled(fixedRate = 1000)
public void faireQuelqueChoseFixedRate() {
    DireBonjour.LOG.info("Hello");
    // action qui dure 3mn
}
```

- Le prochain lancement de la tâche se fera dans 1s
 - Sauf si l'action bloque la thread courante (car les tâches sont synchrones par défaut)

Tâches - Scheduling

- Vous pouvez indiquer un **initialDelay** en ms
 - ➡ Il n'est pris en compte que pour le premier lancement

```
@Scheduled(initialDelay = 5000, fixedDelay = 1000)
public void faireQuelqueChoseFixedDelay() {
    DireBonjour.LOG.info("Hello");
    // action qui dure 3mn
}
```

- Première tâche lancée après 5s
- Seconde lancée après 3mn+1s

Tâches - Scheduling

- Pour les mécaniques plus complexes, vous ferez usage de l'écriture CRON
 - Attention : contrairement à un cron classique, il y a **les secondes** et il n'y a pas d'utilisateur user ou tâche
 - Donc 6 valeurs :
 - ❑ seconde - minutes - heures - jours du mois - mois - jour de la semaine
- Exemples :
 - "0 0 * * * *" = toutes les heures de tous les jours.
 - "*/10 * * * * *" = toutes les 10 secondes.
 - "0 0 8-10 * * *" = 8, 9 et 10 heures tous les jours
 - "0 0 6,19 * * *" = 6:00 du matin et 19:00 du soir tous les jours.
 - "0 0/30 8-10 * * *" = 8:00, 8:30, 9:00, 9:30, 10:00 et 10:30 tous les jours.
 - "0 0 9-17 * * MON-FRI" = toutes les heures de 9h à 17h tous les jours ouvrés
 - "0 0 0 25 12 ?" = tous les minuits de Noël

Tâches - Scheduling

➤ Exemple CRON Spring

```
@Scheduled(cron = "*/1 * * * *")  
public void faireQuelqueChose () {  
    DireBonjour.LOG.info("Hello");  
    // action qui dure 3mn  
}
```

➤ Lancée toutes les secondes

- ➡ Sauf si l'action qui dure 3mn bloque le thread courant

Tâches - Scheduling

- Attention : pour activer l'annotation `@Scheduled` il faut le préciser dans sa configuration

```
@Configuration
@ComponentScan("mon.package.avec.mes.taches")
@EnableScheduling
public class SpringConfiguration { ... }
```

```
<context:component-scan base-package=" mon.package.avec.mes.taches " />
```

```
<task:annotation-driven    executor="taskExecutor"
                           scheduler="threadPoolTaskScheduler" />
```

```
<!-- Vous n'avez pas obligation de declarer un taskExecutor / threadPoolTaskScheduler -->
```

Tâches - Scheduling

- Vous pouvez aussi récupérer les informations de paramétrage de vos tâches via EL du Spring
 - Notez que la propriété se termine par String sauf dans le cas du cron (qui en est déjà une)

```
@Scheduled(fixedDelayString = "${ma.clef.dans.mon.properties}")  
public void faireA() { ...}
```

```
@Scheduled(fixedRateString = "${ma.clef.dans.mon.properties}")  
public void faireB() { ...}
```

```
@Scheduled(cron = "${ma.clef.dans.mon.properties}")  
public void faireC() { ...}
```

Tâches - Scheduling



- Vous pouvez aussi paramétrer vos tâches sans l'annotation et directement dans le XML

```
<task:scheduled-tasks>
  <task:scheduled ref="beanA" method="methodA" fixed-delay="5000" initial-delay="1000" />
  <task:scheduled ref="beanB" method="methodB" fixed-rate="5000" />
  <task:scheduled ref="beanC" method="methodC" cron="*/5 * * * * MON-FRI" />
</task:scheduled-tasks>
```

Tâches - Scheduling



- Pour récupérer la main sur le paramétrage du Scheduling, vous devrez réaliser une classe en `@Configuration` qui implémente l'interface **SchedulingConfigurer**

```
@Configuration
@EnableScheduling
public class SpringConfiguration implements SchedulingConfigurer {

    @Override
    public void configureTasks(ScheduledTaskRegistrar pTaskRegistrar) {
        // Lien avec mon scheduler
        pTaskRegistrar.setTaskScheduler(this.taskScheduler());
        // Ajout de tâches
    }

    @Override
    public Executor getAsyncExecutor() {
        return new ThreadPoolTaskExecutor();
    }

    @Bean
    public TaskScheduler taskScheduler() {
        return new ThreadPoolTaskScheduler();
    }
}
```

Méthodes - **Asynchrone**

- Il est possible de rendre une méthode asynchrone, en ajoutant l'annotation **@Async**
- Vous pouvez paramétrer cette annotation avec l'id de votre *executor*
 - Si vous en avez explicitement déclaré un

```
@Scheduled(...)  
@Async("idExecutor")  
public void faireQuelqueChose () { ... }
```

Méthodes - **Asynchrone**

- Attention : pour activer l'annotation `@Async` il faut le préciser dans sa configuration

```
@Configuration
@ComponentScan("mon.package.avec.mes.classes.annotatees")
@EnableAsync
public class SpringConfiguration { ... }
```

```
<context:component-scan base-package=" mon.package.avec.mes.classes.annotatees " />
```

```
<task:annotation-driven    executor="taskExecutor"
                           scheduler="threadPoolTaskScheduler" />
```

```
<!-- Vous n'avez pas obligation de declarer un taskExecutor / threadPoolTaskScheduler -->
```

Méthodes - **Asynchrone**

- Une méthode asynchrone peut retourner un élément
 - Simple
 - `java.util.concurrent.Future<?>`
 - `java.util.concurrent.CompletableFuture<?>` (Java 8+)
 - `org.springframework.util.concurrent.ListenableFuture<?>` (Spring 4.2+)

- Remarque :
 - Vous ne pouvez pas faire usage de `@Async` et `@PostConstruct`

Méthodes - **Asynchrone**

- La gestion des exceptions pour les méthodes asynchrones qui ne retourneront pas de résultat se fera à part
- Il faudra créer une classe qui implémente l'interface **AsyncUncaughtExceptionHandler**

Méthodes - Asynchrone

➤ Exemple

```
public class MyAsyncUncaughtExceptionHandler implements AsyncUncaughtExceptionHandler {  
  
    @Override  
    public void handleUncaughtException(Throwable ex, Method method, Object... params) {  
        // handle exception  
    }  
}
```

Tâches - Asynchrone



- Pour récupérer la main sur le paramétrage de l'asynchrone et y brancher votre gestion d'erreur, vous devrez réaliser une classe en `@Configuration` qui implémente l'interface **AsyncConfigurer**

```
@Configuration
@EnableAsync
public class SpringConfiguration implements AsyncConfigurer {

    @Bean
    public TaskExecutor taskExecutor() {
        ThreadPoolTaskExecutor result = new ThreadPoolTaskExecutor();
        result.setCorePoolSize(5);
        result.setMaxPoolSize(10);
        result.setQueueCapacity(25);
        return result;
    }

    @Override
    public Executor getAsyncExecutor() {
        return this.taskExecutor();
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return new MyAsyncUncaughtExceptionHandler();
    }
}
```

Tâches - Asynchrone



- En XML la déclaration et l'injection du Handler se font sur le *task:annotation-driven*

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:task="http://www.springframework.org/schema/task"
  xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/task http://www.springframework.org/schema/task/spring-task.xsd">

  <context:component-scan base-package="mes.taches, mon.async.handler" />
  ...
  <task:annotation-driven executor="taskExecutor"
    scheduler="threadPoolTaskScheduler"
    exception-handler=" myAsyncUncaughtExceptionHandler" />
</beans>
```

Travaux pratiques

➤ Exo 25

- On veut enclencher une tâche toutes les secondes
 - ☐ Elle affiche un petit log "-->"
 - ☐ Fait un Thread.sleep de 5s
 - ☐ Elle affiche un petit log "<--"
 - ☐ Elle ne retourne rien
- Ajoutez la gestion des erreurs
- Attention : la tâche devra être asynchrone.



FactoryBean

- Interface, `FactoryBean<T>`, qui va indiquer que votre Bean est une factory
 - Il faudra déclarer votre factory bean comme un Bean
- Trois méthodes
 - **`T getObject()`** : pour fabriquer l'objet retourné par la factory
 - **`Class<?> getObjectType()`** : qui va indiquer la class de l'objet fabriqué
 - **`boolean isSingleton()`** : qui va indiquer si l'objet fabriqué doit être considéré comme un singleton. Cette méthode est implémentée par défaut et retourne true.

FactoryBean

➤ Exemple

```
@Component
public class MyEncryptionFactoryBean implements FactoryBean<EncryptionService> {
    private String algorithm;

    @Override
    public EncryptionService getObject() throws Exception {
        // Complex logic: building a service based on an external algorithm key
        return new EncryptionService(algorithm);
    }

    @Override
    public Class<?> getObjectType() {
        return EncryptionService.class;
    }

    public void setAlgorithm(String algorithm) {
        this.algorithm = algorithm;
    }
}
```

FactoryBean

➤ Injection :

- Il ne faut pas injecter la Factory, mais l'objet fabriquée par la factory, Spring fera le reste

@Component

```
public class SecurityManager {
```

```
    private final EncryptionService encryptionService;
```

```
    // Spring voit que vous avez besoin d'un EncryptionService, il trouve la FactoryBean,  
    // et injecte l'objet en passant par la méthode getObject() de la factory
```

```
    @Autowired
```

```
    public SecurityManager(EncryptionService encryptionService) {
```

```
        this.encryptionService = encryptionService;
```

```
    }
```

```
}
```

Resource

- L'interface `org.springframework.core.io.Resource` du Spring permet de 'remplacer' la classe `File` du Java
- Exemple de classes associées :
 - **ClassPathResource**: Pour les fichiers dans le classpath (ex:src/main/resources)
 - **FileSystemResource**: Pour les fichiers du file système.
 - **UrlResource**: Pour un fichier accédé sur HTTP ou FTP.
 - **ServletContextResource**: Pour un fichier dans le context root de votre Web App.

Resource

➤ Exemple d'utilisation directe

```
@Component
public class MyFileReader {

    // Fichier dans le classpath src/main/resources/data.txt
    @Value("classpath:data.txt")
    private Resource classpathFile;

    // Sur le file système
    @Value("file:C:/config/settings.xml")
    private Resource filesystemFile;

    // Sur un URL
    @Value("https://example.com/api/info.json")
    private Resource webFile;

    public void readFile() throws IOException {
        InputStream is = classpathFile.getInputStream();
    }
}
```

Resource

➤ Exemple d'utilisation via le ResourceLoader

```
@Service
public class DynamicResourceService {

    @Autowired
    private ResourceLoader resourceLoader;

    public boolean doesExist(String fileName) {
        Resource resource = resourceLoader.getResource("classpath:" + fileName);
        return resource.exists();
    }
}
```

Resource

- L'interface Resource a pour objectif premier la lecture des flux
 - Même si FileSystemResource sait aussi écrire via son `getOutputStream()`
- C'est elle qui est utilisée lors du chargement des fichiers de configuration XML
 - Voir les exemples dans le support associé

Evènements

- Vous pouvez générer ou rattraper des évènements
- Le framework Spring aussi
- Il existe des mécaniques simples permettant de se brancher sur tel ou tel évènement en Spring
- Le pattern est celui du Listener / Publisher
 - Avec ou sans Annotations (privilégiez avec dans les versions > 4.2 du Spring)

Evènements - Avec Annotations

- Faire un Listener en Spring = un objet qui est à **l'écoute** d'évènements
 - Annoter la classe en **@Component** (la placer dans un *component scan*)
 - Annoter la méthode avec **@EventListener**
 - ❑ L'annotation peut porter des conditions
 - Choisir **le paramètre** en fonction **du type** de l'évènement ciblé
 - ❑ Certains évènements peuvent supporter plusieurs paramètres

Evènements - Avec Annotations

➤ Exemples

```
@Component
public class MonEventListenerA {

    @EventListener
    public void handle(BrokerAvailabilityEvent event) {
        // Faire qqc
    }
}
```

```
@Component
public class MonEventListenerB {

    @EventListener(condition = "#event.brokerAvailable")
    public void handleSuccessful(BrokerAvailabilityEvent event) {
        // Faire qqc
    }
}
```

Evènements - Avec Annotations

➤ Pour générer un évènement

- Annoter la classe en **@Component** (la placer dans un *component scan*)
- Annoter la méthode avec **@EventListener**
- Choisir **le retour** en fonction **du type** de l'évènement ciblé

Evènements - Avec Annotations

➤ Exemples (avec annotations)

```
@Component  
public class MonEventListenerA {  
  
    @EventListener  
    public MonEvent handle() {  
        // Faire qqc et renvoyer un objet  
    }  
}
```

Evènements - Sans Annotation

- Pour faire un Listener en Spring, seconde approche sans l'annotation
 - Annoter la classe en **@Component** (la placer dans un *component scan*)
 - Elle doit implémenter l'interface `ApplicationListener<E>`
 - ❑ **public void onApplicationEvent(E event)**
 - Ecrire la méthode associée à l'interface, la typer en remplaçant *E* par votre classe d'event

Evènements - Sans Annotation

➤ Exemples

```
@Component
public class CustomSpringEventListener
    implements ApplicationListener<BrokerAvailabilityEvent> {

    @Override
    public void onApplicationEvent(BrokerAvailabilityEvent event) {
        System.out.println(event.getMessage());
    }
}
```

Evènements - Sans Annotation

- Pour générer un évènement, seconde approche sans l'annotation
 - Annoter la classe en **@Component** (la placer dans un *component scan*)
 - Ajouter un attribut **@Autowired** de type **ApplicationEventPublisher**
 - Quand bon nous semble, faire usage de la méthode **publishEvent**
- Remarque : vous pouvez utiliser les évènements du Spring ou créer de simple POJO
 - Depuis Spring 4.2, il n'y a pas de contrainte d'héritage.
 - Avant Spring 4.2 il faut obligatoirement que vos *event* héritent de la classe **ApplicationEvent**

Evènements - Sans Annotation

- Autre technique sans les annotations

```
@Component  
public class MyComponent {  
  
    @Autowired  
    private ApplicationEventPublisher publisher;  
  
    public void envoyerEvn() {  
        // ....  
        this.publisher.publishEvent(new MonEvent());  
    }  
  
}
```

Evènements - Déjà existants

- Chaque module du spring peut venir avec ses évènements, en voici quelques-uns qui font partie du Spring Core et qui sont générés par l'ApplicationContext
- Pour des raisons de rétrocompatibilité, tous héritent de *org.springframework.context.ApplicationEvent*
 - **ContextRefreshedEvent** : lors de l'initialisation ou le rafraîchissement de l'ApplicationContext
 - **ContextStartedEvent** : lors du démarrage de l'ApplicationContext
 - **ContextClosedEvent** : lors de la fermeture de l'ApplicationContext
 - **ContextStoppedEvent** : lors de l'arrêt de l'ApplicationContext
 - **RequestHandledEvent** : En web, lorsque une HttpRequest est gérée par l'ApplicationContext

Internationalisation

- Spring prend en charge la gestion de vos messages en plusieurs langues
 - C'est une simplification de la classe *ResourceBundle* du Java classique
- On se place ici hors contexte Web, car en général chaque framework web vient avec sa solution de multilingue

Internationalisation

- Dans un premier temps, vous devrez créer vos fichiers de ressources multilingues
 - En général on les place dans le dossier **src/main/resources**
 - ❑ Ou un sous dossier
 - Le nom du fichier est libre, mais
 - ❑ En général il porte le nom **message**
 - ❑ Il faut plus le voir comme un **préfixe**
 - L'extension du fichier sera un **.properties**
 - Il peut y avoir plusieurs fois le même fichier mais avec un **suffixe** qui représente sa locale (**langue**)
 - ❑ Format xx_yy (xx = lang, optionnel yy = country)
 - Ce qui donnera par exemple :
 - message_fr_CA.properties
 - message_en.properties

Internationalisation

➤ Dans les fichiers vous indiquerez vos messages sous la forme

➤ **clef = valeur** : il est conseillé d'utiliser un nom de clef en anglais

```
error.wrong.password=Mauvais mot de passe  
error.user.unknown=Utilisateur inconnu  
error.technical=Erreur technique {0}
```

➤ Attention à l'encodage du fichier properties

➤ {x} représente un trou dans le message

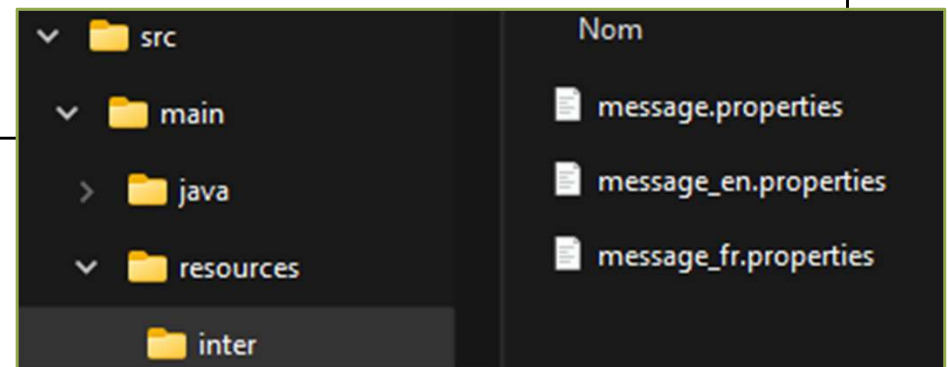
Internationalisation

- Dans votre code Java vous allez déclarer un @Bean de type
`org.springframework.context.support.ResourceBundle`
MessageSource
- Il faudra simplement lui indiquer où se trouvent vos fichiers de messages
 - En utilisant la méthode *setBasename*

Internationalisation

➤ Déclaration du bean

```
@Component  
public class MyComponent {  
  
    @Bean  
    public ResourceBundleMessageSource messageSource() {  
        var source = new ResourceBundleMessageSource();  
        source.setBasename("inter/message");  
        return source;  
    }  
}
```



Internationalisation

- Là où vous avez besoin de vos messages, **injectez** votre bean et faites appel à sa méthode *getMessage*

```
public static class MyBean {  
    @Autowired  
    private ResourceBundleMessageSource messageSource;  
  
    public void doSomething () {  
        var msg = messageSource.getMessage("error.technical",  
                                           new Object[]{"Joe"},  
                                           Locale.getDefault());  
    }  
}
```

Caches

- Vous pouvez demander au Spring de mettre en cache le résultat d'une méthode
 - Peut vous éviter un appel à la base de données, par exemple
- L'activation du cache commencera par l'utilisation de l'annotation **@EnableCaching** sur une de vos classes de configuration

```
@SpringBootApplication
@EnableCaching
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Caches

- Côté dépendances, si vous êtes en Spring Boot

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache-test</artifactId>
  <scope>test</scope>
</dependency>
```

Caches

- Dans votre classe `@Service`, sur la classe elle-même
 - **@CacheConfig** : pour centraliser les paramétrages des autres `@CacheXxx`
- Dans votre classe `@Service`, sur vos méthodes, vous pourrez utiliser
 - **@Cacheable** : pour indiquer que vous voulez mettre le résultat de l'appel à la méthode en cache
 - ❑ ATTENTION : `org.springframework.cache.annotation.Cacheable` et pas `jakarta.persistence.Cacheable`
 - **@CachePut** : pour indiquer que vous voulez mettre à jour la valeur actuellement dans le cache
 - **@CacheEvict** : pour retirer une valeur du cache

Caches

➤ Exemple

Sera rangé sous la clef
books::id

Car cacheNames = "books"

```
@CacheConfig(cacheNames = "books")
@Service
public class BookService {

    @Cacheable(key = "#id")
    public Book findById(Long id) {
        return repository.findById(id);
    }

    @CachePut(key = "#book.id")
    public Book save(Book book) {
        db.put(book.getId(), book);
        return book;
    }

    @CacheEvict(key = "#id")
    public void delete(Long id) {
        repository.deleteByld(id);
    }
}
```

Caches

- Les annotations peuvent aussi faire usage du résultat
 - ➡ Mot clef **result** en SpEL
- Mais attention, dans le cas de `@Cacheable` il a besoin de la clef avant d'appeler la méthode, vous devrez faire usage de **unless**

```
@Cacheable(key = "#email", unless = "#result == null || #result.getId() == null")  
public User findByEmail(String email) {  
    return repository.findByEmail(email);  
}
```

Caches

- En Spring Boot, vous pouvez indiquer des paramétrages plus globaux
 - **spring.cache.type** : pour indiquer l'implémentation du cache
 - **spring.cache.cache-names** : noms des caches séparés par une ',' qui seront préparés au démarrage de l'application
 - **spring.cache.*** : propriétés spécifiques en fonction de l'implémentation du cache
- Par défaut, le cache sera mémoire
 - Celui du Java : ***ConcurrentMapCacheManager***



Caches - Redis

- Redis est un cache très répandu aujourd'hui
 - C'est un serveur ↔ quelque chose d'externe à votre application
- Il est intégrable simplement depuis Spring Boot 4.1
 - Via la dépendance **spring-boot-starter-data-redis**
- Exemple de configuration pour Redis

```
# Pour accéder au serveur Redis (c'est à vous de le démarrer avant votre application)
spring.data.redis.host=localhost
spring.data.redis.port=6379

# Clefs classiques de configuration
spring.cache.type=redis
# Mes noms de clefs seront TOUS prefixés par app::
spring.cache.redis.key-prefix=app::
spring.cache.redis.use-key-prefix=true
spring.cache.redis.time-to-live=10m
```



Caches - Redis

- L'utilisation de Redis comme cache vous permet d'utiliser deux objets supplémentaires le
 - **RedisTemplate** : pour `<Object, Object>`
 - **StringRedisTemplate** : pour `<String, String>`
- Spring Boot les configure automatiquement
 - Vous pouvez les court-circuiter si besoin (souvent le cas)
- Par défaut, ces deux objets vont faire usage d'une sérialisation binaire
 - Les objets que vous mettrez en cache **DEVRONT OBLIGATOIREMENT** implémenter **`java.io.Serializable`**
 - Ils ne seront pas agréablement lisibles dans le CLI redis (sur le serveur Redis)

Caches - Redis



➤ Pour pallier les deux défauts ⇔ court-circuitons SB

```
@Configuration
@EnableCaching
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory connectionFactory) {
        var template = new RedisTemplate<String, Object>();
        template.setConnectionFactory(connectionFactory);

        // Nous sommes en Spring Boot 4.1 ici, nous utiliserons le Jackson 3
        // Le validateur autorise les classes de notre package com.banque ainsi que
        // les types java.* et java.time.* utilisés dans les DTOs
        PolymorphicTypeValidator ptv = BasicPolymorphicTypeValidator.builder().allowIfBaseType(Object.class).build();

        var jackson3Mapper = JsonSerializer.builder().disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
            // Active l'écriture de @class dans le JSON pour permettre la désérialisation
            // correcte du type concret lors de la lecture depuis Redis
            .activateDefaultTyping(ptv, DefaultTyping.NON_FINAL).build();

        var jsonSerializer = GenericJacksonJsonRedisSerializer.builder(() -> jackson3Mapper.rebuild()).build();

        // Nous utiliserons une sérialisation JSON plutôt que binaire
        template.setKeySerializer(new StringRedisSerializer());
        template.setHashKeySerializer(new StringRedisSerializer());

        template.setValueSerializer(jsonSerializer);
        template.setHashValueSerializer(jsonSerializer);

        return template;
    }
}
```

Caches - Redis



- A partir du moment où vous faites du Redis + JSON, vous aurez les dépendances suivantes en Spring Boot

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<!-- Pour Json -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```



Caches - Redis

- Dans cette approche
 - **RedisTemplate**<Object, Object> : n'existe plus
 - **StringRedisTemplate**<String, String> : existe toujours et a été initialisé par Spring Boot
 - **RedisTemplate**<String, Object> : notre Bean à nous que nous pourrions utiliser

- MAIS, **ATTENTION**, *@Cachexxx* N'EST PAS branché sur notre RedisTemplate



Caches - Redis

- Il faut donc faire le lien entre notre CacheManager et notre template

```
@Bean
public RedisCacheManager cacheManager(RedisConnectionFactory connectionFactory) {

    var keySerializer = (RedisSerializer<String>) this.redisTemplate(connectionFactory).getKeySerializer();
    var valueSerializer = (RedisSerializer<Object>) this.redisTemplate(connectionFactory).getValueSerializer();

    var cacheConfig = RedisCacheConfiguration.defaultCacheConfig()
        .serializeKeysWith(RedisSerializationContext.SerializationPair.fromSerializer(keySerializer))
        .serializeValuesWith(RedisSerializationContext.SerializationPair.fromSerializer(valueSerializer));

    return RedisCacheManager.builder(connectionFactory).cacheDefaults(cacheConfig).build();
}
```

- De même, si nous avons fait usage de clefs de paramétrage, c'est à **NOUS** de les faire passer au cacheManager (TTL, prefix ...)



Caches - Redis

➤ Utilisation : @CacheXxx ⇔ Rien ne change

Même si nos clefs sont des Integer, ils seront transformés en String de part notre configuration

```
@Service
@CacheConfig(cacheNames = "products")
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    @Cacheable(key = "#id")
    public Product getProductById(Integer id) {
        return repository.findById(id);
    }

    @CachePut(key = "#product.id()")
    public Product updateProduct(Product product) {
        return repository.save(product);
    }

    @CacheEvict(key = "#id")
    public void deleteProduct(Integer id) {
        return repository.delete(id);
    }
}
```

Caches - Redis



- La mise en place d'un cache Redis **n'est pas anodine**
- C'est un serveur de structures de données mémoire ultra-rapide.
 - le RedisTemplate vous permet de piloter Redis directement pour exploiter ses fonctionnalités avancées :
- Par exemple vous pouvez manipuler :
 - Les compteurs atomiques : Incrémenter un nombre de manière ultra-sécurisée, même si 10 000 utilisateurs cliquent en même temps (impossible à faire proprement avec une base de données classique ou @Cacheable).
 - Les structures complexes : Manipuler des Listes (files d'attente), des Sets (listes d'éléments uniques) ou des Hashes.
 - La gestion fine du temps : Définir une durée de vie (TTL) différente pour chaque clé selon le contexte (ex: un jeton de connexion qui expire dans 15 minutes).



Caches - Redis

- Pour faire explicitement usage de notre template, nous devons le déclarer et l'injecter

```
@Service
public class ProductService {

    // Notre Template
    private final RedisTemplate<String, Object> redisTemplate;

    private final ProductRepository repository;

    // Injection de notre template
    public ProductService(RedisTemplate<String, Object> redisTemplate, ProductRepository repository) {
        this.redisTemplate = redisTemplate;
        this.repository = repository;
    }
}
```

Attention notre template peut 'manquer' certains éléments de configuration qu'il nous faudra appliquer directement lors de son utilisation



Caches - Redis

➤ Maintenant, nous pouvons l'utiliser

➡ Compteur Atomique

```
public Long incrementerLecturesArticle(Long articleId) {  
    String key = "article::vues::" + articleId;  
    // .increment() est atomique au niveau de Redis : aucun risque de doublon ou de conflit de thread  
    return redisTemplate.opsForValue().increment(key);  
}
```



Caches - Redis

➔ Manipulation de structures (Map) et de TTL

```
public void ajouterAuPanier(Long utilisateurId, String produitId, Integer quantite) {  
    String key = "panier::utilisateur::" + utilisateurId;  
  
    // On stocke ou on met à jour une entrée spécifique dans le Hash Redis  
    redisTemplate.opsForHash().put(key, produitId, quantite);  
  
    // On en profite pour dire que le panier expire au bout de 24h d'inactivité  
    redisTemplate.expire(key, 24, TimeUnit.HOURS);  
}  
  
public Map<Object, Object> recupererPanier(Long utilisateurId) {  
    String key = "panier::utilisateur::" + utilisateurId;  
    // Récupère tout le contenu du panier d'un coup  
    return redisTemplate.opsForHash().entries(key);  
}
```

Attention aux options
spring.cache.redis.key-prefix
si vous en avez fait usage
C'est à vous de les appeler

Attention aux options
spring.cache.redis.time-to-live
si vous en avez fait usage
C'est à vous de les appeler

Actuator

- Le module Spring Boot Actuator permet de monitorer en temps réel ses applications
- Il fait usage du standard Micrometer
 - <https://micrometer.io/>
- Avec ce module, vous pouvez *exporter* des métriques vers différents outils de monitoring comme :
 - Amazon CloudWatch, Azure Monitor, Google Cloud Monitoring (Stackdriver), Datadog, New Relic, Dynatrace.
 - InfluxDB, Graphite, OpenTSDB, KairosDB, TimescaleDB.
 - AppOptics, Atlas, Elastic, Humio, JMX, Wavefront.

Actuator

- Selon l'outil de monitoring que vous allez indiquer à Spring Boot (via une dépendance)
 - Mode **Push** : Votre application Java va envoyer des informations à l'outil de monitoring
 - ❑ Ex: Influx
 - Mode **Pull** : Votre outil de monitoring va se brancher sur votre application Java afin d'y collecter les informations de monitoring
 - ❑ Ex: Prometheus

Actuator

➤ Côté dépendances on verra toujours :

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

➤ Puis, une autre dépendance en fonction de votre outil de monitoring :

➤ Exemple : Influx

```
<dependency>  
  <groupId>io.micrometer</groupId>  
  <artifactId>micrometer-registry-influx</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

➤ Exemple : Prometheus

```
<dependency>  
  <groupId>io.micrometer</groupId>  
  <artifactId>micrometer-registry-prometheus</artifactId>  
  <scope>runtime</scope>  
</dependency>
```



Prometheus

- Projet open source de monitoring applicatif
 - <https://prometheus.io/>
- L'objectif est que votre application Java laisse à disposition des informations (de monitoring) qui seront récupérables par votre serveur Prometheus (externe à votre projet)
- Du coup, c'est le Serveur Prometheus qui sait trouver votre application Java (et pas le contraire)
 - Approche *Pull*



Prometheus

- Côté projet Java, vous devrez indiquer le / les end-point que vous souhaitez voir exposer par Spring Boot via la propriété :
 - **management.endpoints.web.exposure.include=health,info,metrics,prometheus**
 - ❑ **health** : permet de savoir si votre application tourne correctement
 - ❑ **info** : expose les informations générales sur votre application (nom, version, ...)
 - ❑ **metrics** : contient les informations techniques de la JVM
 - ❑ **prometheus** : contient les informations au format prometheus
- Il est parfois nécessaire d'ajouter les règles d'accès via les clefs
 - **management.endpoints.access.default=unrestricted**
 - **management.endpoint.info.access=unrestricted**



Prometheus

- Tous les endpoint se retrouvent sur leurs URLs respectifs
 - Health et info n'ont pas besoin de prometheus, on les retrouve sur leurs URLs
 - ❑ <http://xxx:8080/context-root/actuator/health>
 - ❑ <http://xxx:8080/context-root/actuator/info>
 - Prometheus vous donne un front graphique



Prometheus

➤ Côté serveur Prometheus

- Vous devrez paramétrer son fichier de configuration afin qu'il interroge votre application Java

```
global:
  scrape_interval: 10s # How frequently to scrape targets by default

scrape_configs:
  - job_name: 'spring_micrometer' # The job name is assigned to scraped metrics by default.
    metrics_path: 'banque/actuator/prometheus' # The HTTP resource path on which to fetch metrics from targets.
    scrape_interval: 5s # How frequently to scrape targets from this job.
    static_configs: # A static_config allows specifying a list of targets and a common label set for them
      - targets: ['java.app:8080']
  #
  # banque : context root of the Java app
  # java.app : container hostname (or machine name) for the Java app !!! DO NOT USE '_' with the name
  #       You can use host.docker.internal with docker
  # 8080 : port of the Java app
  #
```

prometheus.yml



Prometheus

- Côté code Java, par défaut, **vous n'avez rien à faire** sauf si vous voulez ajouter des métriques spécifiques
- Pour customiser les informations (*endpoint* info), ajoutez simplement les éléments suivant dans votre fichier properties
 - ➡ Tous les **info.app** seront disponibles

```
management.endpoint.info.enabled=true  
management.info.env.enabled=true
```

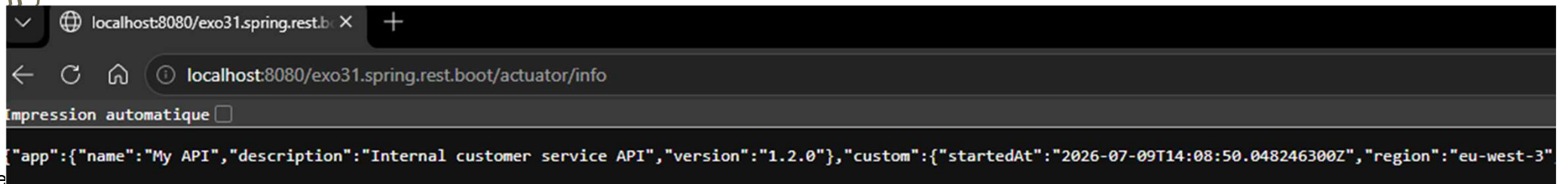
```
info.app.name=My API  
info.app.description=Internal customer service API  
info.app.version=1.2.0
```



Prometheus

- Toujours pour info, si vous avez besoin de fabriquer dynamiquement des informations
 - Créez une classe qui implémente `InfoContributor`

```
@Component
public class MyInfoContributor implements InfoContributor {
    @Override
    public void contribute(Builder builder) {
        builder.withDetail("custom",
            Map.of("service", "billing-service", "region", "eu-west-3", "startedAt", Instant.now().toString()));
    }
}
```





Prometheus

➤ Si vous souhaitez compléter vos métriques, vous pouvez faire usage de l'annotation **@Observed**

➤ Il faudra ajouter la dépendance

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <!-- Avant: <artifactId>spring-boot-starter-aop</artifactId> -->
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

➤ Et le bean

```
@Configuration(proxyBeanMethods = false)
public class ObservationConfig {
    @Bean
    public ObservedAspect observedAspect(ObservationRegistry observationRegistry) {
        return new ObservedAspect(observationRegistry);
    }
}
```



Prometheus

➤ Dans votre code

```
@GetMapping("/lister")
@Observed(name = "rest.comptes.lister", contextualName = "rest#comptes#lister")
public ResponseEntity<List<CompteDtoOut>> listerCompte(@RequestParam(value = "userId") int userId)
    throws FonctionnelleException, ErreurTechniqueException {
```

- name : le nom de la métrique
- contextualName : nom du span qui apparaîtra dans l'outil de tracing (Jaeger/Zipkin)

➤ Dans votre Prometheus :

